

Seminar Thesis on *Generating Computational Cognitive Models using Large Language Models*

Submitted by Christian Block and YingZhu Chen

Contents

1	Executive Summary	1
2	Motivation	2
3	GeCCo’s Proposed Pipeline	2
4	Experiments	3
4.1	Experiment 1: Decision Making	3
4.2	Experiment 2: Learning	3
4.3	Experiment 3: Planning	3
4.4	Experiment 4: Working Memory	4
5	Control Experiments	4
6	Discussion	5
6.1	Connection to other Papers in the Seminar	6

Executive Summary

The pipeline proposed in *Generating Computational Cognitive Models using Large Language Models* (GeCCo) uses Large Language Models (LLMs) to automatically generate, fit, and iteratively refine computational cognitive models as Python functions. Given a task description, participant behavioural data, and a template function, the pipeline in GeCCo prompts an LLM to propose candidate models, fits those proposals to held-out data, and refines them based on feedback derived from predictive performance.

The approach was benchmarked across four cognitive domains—decision making, learning, planning, and memory—using three open-source LLMs of varying parameter size, capacity, and training procedure. Across the four benchmark domains, the strongest LLM-generated models significantly outperformed hand-crafted baselines in decision making, learning, and working memory. In the planning domain, performance was numerically better but did not reach conventional statistical significance, so the overall result is best described as broadly competitive rather than uniformly superior performance.

Motivation

Computational cognitive models provide a formal framework for understanding human behaviour. In practice, researchers hand-craft these models, fit them to behavioural data, and iteratively refine them. This process is slow, skill-intensive, and systematically biased due to a person’s past exposure to the cognitive literature. Translating a psychological theory into a working algorithm requires lengthy literature reviews and repeated coding cycles. Researchers must simultaneously act as domain experts in cognitive science, theoreticians for data modeling, and software engineers for implementation. Perhaps most importantly, researchers tend to explore only the model classes they already know and favour, potentially missing alternative explanations for the data. This resembles the *streetlight effect*, or what Kaplan (1964) called the principle of the drunkard’s search: investigation is pulled toward the places where existing tools and theories make searching easiest. The aforementioned limitations narrow the space of models considered, motivating an automated approach that is less tied to any single school of thought in theoretical cognitive science.

Three specific LLM capabilities make them well-suited for proposing cognitive models for experimental data.¹ First, LLMs can process task descriptions and behavioural data written in natural language, which makes them highly adaptable across different experimental paradigms without the need to provide a custom interface. Second, using broad representational and reasoning abilities, they can identify structure in complex behavioural data and generate plausible psychological hypotheses by reasoning over task structure and behavioural patterns. Third, LLMs can translate those hypotheses directly into executable Python functions. These three abilities have already been utilized in automated statistical modelling through probabilistic program generation, and GeCCo applies the same logic to cognitive science.

GeCCo’s Proposed Pipeline

Each pipeline run consists of 10 sampling iterations, repeated across 5 independent runs per beforementioned cognitive domain. The process begins by providing the base LLM with a fixed prompt containing a natural language task description, a subset of behavioral data, strict instructions, a domain-specific code template, and starting from the second iteration onwards, iterative feedback. Using this information, the LLM generates three unique cognitive models, ensuring that each features a non-overlapping set of parameters.

Once generated, these models are parsed from the response, converted into executable Python functions, and have their specific parameter names extracted. They are then fitted to a held-out validation dataset that was deliberately excluded from the initial prompt. To prevent the models from getting stuck in local minima, this fitting process is initialized from 20 random starting points.

After the fitting stage, the models are evaluated using the Bayesian Information Criterion

¹The models used were: Llama-3.1-Instruct-70B (Dubey et al., 2024), DeepSeek-R1-Distill-Llama-3.1-70B (Guo et al., 2025), and Qwen2.5-72B-Instruct (Yang et al., 2024).

(BIC) to identify the single best-performing candidate among the three. Finally, the code for this winning model, along with a list of previously used parameter names, is gathered to act as feedback. This feedback is then injected into the prompt for the next iteration, effectively guiding the LLM’s future proposals and preventing it from duplicating its past ideas.

Experiments

Two complementary metrics are used throughout the evaluation. BIC balances goodness of fit against model complexity by penalising the number of free parameters; when two models fit equally well. The Exceedance Probability (EXP) is used in the final evaluation stage and quantifies the posterior probability that a given model provides the most prevalent explanation of observed behaviour across the population, translating raw BIC scores into an interpretable confidence measure.

Experiment 1: Decision Making

People were asked to choose between two fake products (Option A and Option B). They were given ratings from four different "experts," but they were also told that some experts were more trustworthy (had higher "validity") than others. The baseline was the probabilistic Weighted Additive Decision model, which uses four separate parameters to weight the four features. The Llama-generated model achieved a substantially lower BIC (39.40 vs. 51.97; $p < .001$). Rather than requiring four parameters, the LLM reduced the problem to a single *discount factor* that smoothly interpolates between three classic decision strategies—*Take the Best*, *Equal Weighting*, and *Weighted Additive*—mirroring complex Bayesian inference but arriving at the solution naturally.

Experiment 2: Learning

Participants played a "slot machine" style game where they repeatedly chose between two options to win money. In a special version of this game, they didn’t just see what they won, but they also saw what the other option would have paid out if they had chosen it. The baseline was a four-learning-rate variant of the Rescorla–Wagner model augmented by a perseveration parameter capturing choice stickiness. The LLM-generated model (BIC 77.97 vs. 85.24; EXP = 0.97) replaced the asymmetric learning rates with a clean separation between actual and counterfactual value traces, and replaced the perseveration parameter with a psychologically motivated *forgetting parameter* that decays unchosen-option values over time, mirroring the limits of working memory.

Experiment 3: Planning

This is a two-step puzzle game. Participants pick a "magic carpet," which usually takes them to a specific mountain. Once at the mountain, they ask a genie for gold. However, sometimes a "strong wind" blows the carpet to the wrong mountain. This game tests whether people just blindly repeat actions that win them gold, or if they actually "plan ahead" by understanding the rules of the wind and the carpets. The baseline was a Hybrid model combining model-free and model-based value iteration, mixed by a free weighting parameter. The R1-generated model achieved a lower mean BIC (432.47 vs. 437.38; EXP = 0.85), although the BIC difference did not reach conventional significance ($p =$

.27). By unifying the model-free/model-based dichotomy into a single system using two transition-dependent learning rates, and by omitting the temporal discounting parameter, the model aligned with recent theoretical calls to move beyond strict dual-system accounts of planning.

Experiment 4: Working Memory

People had to learn through trial-and-error which button to press when shown different shapes on a screen. Sometimes they only had to juggle 3 shapes in their head (low cognitive load), and sometimes they had to juggle 6 shapes (high cognitive load). The baseline was the RL-WM model (Collins and Frank, 2012), in which a slow RL module and a fast but capacity-limited WM module operate independently, with a load-dependent mixing weight. The Llama-generated model outperformed the baseline substantially (BIC 267.25 vs. 275.20; EXP = 0.99). Contrary to the standard assumption that reinforcement learning is immune to cognitive load, the LLM’s model assigned separate RL learning rates for low- and high-load conditions (`lr_low` / `lr_high`), suggesting that load modulates the core learning signal—a conclusion consistent with fMRI evidence that working-memory load can strengthen striatal reward-prediction-error signals (Collins et al., 2017).

Control Experiments

A mixed-effects regression predicting BIC from *reasoning ability*, *model family*, and *model size* found that Llama outperformed Qwen ($\beta = 5.624$, $p = .03$) despite Qwen having more parameters, suggesting that training data and protocol matter more than raw scale. Reasoning ability showed only a marginal advantage ($\beta = -0.266$, $p = .07$), falling short of conventional significance.

Systematically removing each prompt component and refitting a mixed-effects model on post-ablation BIC scores revealed that all four components contribute meaningfully, though with different magnitudes. Iterative feedback was the strongest single driver of performance ($\beta = -17.040$, $p < .05$), followed by participant data ($\beta = -12.836$, $p < .05$), task description ($\beta = -9.834$, $p < .05$), and the code template ($\beta = -9.359$, $p < .05$). The dominance of the feedback component confirms that the iterative refinement loop—not merely the quality of the initial prompt—is what separates GeCCo from a single-shot query.

The LogProber method was applied to Llama prompts across all four domains. Acceleration scores were well below the contamination threshold of 1.0 (range: 0.017–0.648), confirming that the LLM is reasoning about novel problems rather than recalling memorised solutions from pretraining.

On synthetic datasets with known generating processes, the LLM successfully reverse-engineered the true model in both learning (Rescorla–Wagner) and decision-making (Take the Best; Tallying) domains, validating the mathematical soundness of the approach.

CENTAUR is a large foundation model trained to predict human behaviour across cognitive tasks, serving as a proxy for the ceiling of explainable variance. GeCCo matched or exceeded CENTAUR’s negative log-likelihood across all four domains, while producing interpretable Python code rather than opaque model weights.

Discussion

The GeCCo pipeline successfully demonstrates that open-source LLMs can act as automated research assistants, generating and refining psychological models using standard Python code. Across four distinct areas of human cognition, these AI-generated models were broadly competitive with the best hand-crafted models built by human experts: they significantly outperformed baselines in three domains, while the planning domain showed a numerical but non-significant advantage. Crucially, the AI did not merely reproduce known ideas; it frequently proposed simpler or alternative theoretical explanations. For example, the LLM discovered a single discount factor for decision-making heuristics, cleanly separated experienced from counterfactual outcomes in learning tasks, and suggested that cognitive load directly modulates the reinforcement learning rate in working memory tasks.

Because human cognitive modeling is often inherently limited by the theoretical assumptions researchers start with, they may overlook simpler explanations. GeCCo partially mitigates this "streetlight effect" by acting as a hypothesis-generation tool that expands the set of possible models, though the final interpretation still relies on human scientific judgment. Rigorous control experiments proved this success was driven by the AI's ability to learn from its mistakes through iterative feedback, rather than from data contamination. Furthermore, by evaluating the generated models on strictly held-out data that the LLM never sees during the prompting phase, the pipeline effectively mitigates the risk of overfitting. The AI's resulting code remained highly interpretable to humans while capturing just as much predictable human behaviour as massive, opaque foundation models like CENTAUR.

A major benefit of GeCCo is its high accessibility; the system works out of the box using standard, open-source AI models without the need for expensive or time-consuming retraining. It utilizes a hybrid approach where the AI acts as an intelligent proposal engine to brainstorm the mathematical structures, but the actual testing of how well those models fit the data is done offline using standard, rigorous statistical tools. Thus, the final model selection is determined by BIC on held-out validation data, while the proposal process remains a hybrid search procedure guided by iterative feedback from previous rounds rather than by unverified AI outputs alone.

Despite these breakthroughs, the authors outline several limitations that chart the course for future research. One limitation is template bias; the AI is currently provided with a basic, human-made code template to guide its syntax, which might unintentionally restrict its creative exploration to specific model classes. Additionally, there is an inherent risk of over-trusting the LLM-generated explanations; because the generated code often looks clean and authoritative, researchers might accept it too quickly. The system's current feedback mechanism relies purely on predictive performance metrics like the BIC. However, a model with a lower BIC is not automatically the best scientific explanation, and human researchers must still verify if the mechanisms are psychologically meaningful and biologically plausible. Future iterations could incorporate a secondary AI to provide qualitative feedback—critiquing the theoretical logic or biological constraints—to further minimize the need for human oversight.

Finally, the study is limited in scope because it evaluated four specific, controlled laboratory tasks. Future research must extend this pipeline to more complex areas, such as

visual perception and natural language processing, which involve richer data formats. It is also vital to test GeCCo’s capacity for cross-task generalization, ensuring that an AI-generated cognitive theory can robustly explain behavior across related tasks rather than just fitting a single dataset. Ultimately, these findings suggest that LLMs have the potential to democratize the highly technical field of computational modeling and significantly accelerate the pace of scientific discovery.

Context Within Related Work

This iterative optimization paradigm closely mirrors the broader industry shift toward recursive self-improvement models and automated research engines, such as Sakana AI’s “The AI Scientist”, Google DeepMind’s autonomous research loops for scientific discovery, and Anthropic’s research into iterative code-refinement. These modern frameworks employ an AI system to continuously write, test, critique, and upgrade its own code to automate the scientific pipeline. GeCCo successfully adapts this concept to the behavioral sciences by turning the LLM into a rapid hypothesis proposal engine. Crucially, whereas generalized self-improvement models risk hallucination or runaway optimization errors when critiquing themselves, GeCCo introduces a vital safeguard: the self-improvement cycle is anchored in a hybrid loop, where standard, rigid statistical frameworks provide the objective ground-truth feedback that drives the AI’s refinement process.

Connection to other Papers in the Seminar

The work by [Rmus et al. \(2025\)](#) introduces a framework for generating computational cognitive models using Large Language Models (LLMs), moving the field from manually handcrafted theories to automated, AI-driven model discovery based on task instructions and behavioral data. This approach is part of a broader paradigm shift in cognitive science and neuroscience, and its relationship to the other provided papers can be understood through several distinct themes: direct methodological parallels, cross-scale model discovery, the expanding role of LLMs in science, and the bridging of top-down and bottom-up neural research.

[Rmus et al. \(2025\)](#) shares the most direct methodological and conceptual parallel with the research by [Castro et al. \(2025\)](#). Both papers tackle the complex challenge of discovering symbolic cognitive models directly from human and animal behavior. While traditional cognitive science relies on human intuition to formalize theories, both Rmus and Castro demonstrate that modern AI algorithms can autonomously propose, fit, and evaluate mathematical models that explain learning and decision-making processes. They collectively establish a new standard for automated theory building in behavioral science.

This theme of AI-driven automated discovery extends beyond behavior down to the level of neural circuits, linking [Rmus et al. \(2025\)](#) to the works of [Lueckmann et al. \(2026\)](#) and [Tilbury et al. \(2025\)](#). While Rmus focuses on high-level cognitive architectures, Lueckmann and Tilbury utilize AI systems and LLM-based tree searches to discover mechanistic models and tuning laws for neural activity. Together, these papers illustrate a unified methodological trend across scales: whether predicting whole-organism behavior or visual cortical neuronal firing rates, AI is increasingly being used to distill complex biological data into parsimonious, interpretable mathematical equations.

Furthermore, [Rmus et al. \(2025\)](#) exemplifies the growing utility of LLMs as active sci-

entific reasoning engines, a concept heavily supported by several other papers in this collection. [Luo et al. \(2025\)](#) demonstrate that LLMs can surpass human experts in predicting the outcomes of neuroscience experiments, proving that these models possess a deep, actionable understanding of the domain. Similarly, [Luppi et al. \(2025\)](#) and [Ghayem et al. \(2026\)](#) utilize LLMs and rich text representations to synthesize vast amounts of complex neuroscience literature, performing meta-analyses and mapping cognitive functions to brain regions. Viewed alongside these works, [Rmus et al.](#) represents the next logical step: upgrading LLMs from passive synthesizers of existing literature to active generators of novel scientific hypotheses and code.

Finally, the top-down cognitive modeling approach of [Rmus et al. \(2025\)](#) provides a necessary complement to the bottom-up neural decoding and foundation models explored in the remaining literature. Papers by [Wang et al. \(2025\)](#), [Ryoo et al. \(2025\)](#), and [Wairagkar et al. \(2025\)](#) focus intensely on raw neural data, utilizing foundation models and hybrid state-space models to decode brain activity for neuroprostheses and real-time predictions. While these papers excel at translating neural spikes into variables like speech or stimuli, they often lack explicit cognitive frameworks. [Rmus et al. \(2025\)](#) provides the computational cognitive architectures that could eventually be mapped onto the neural dynamics decoded by these other models. In summary, [Rmus et al.](#) serves as a crucial behavioral anchor in a scientific landscape that is rapidly adopting AI to map the brain, decode its signals, and discover its fundamental laws.

References

- Castro, P. S., Tomasev, N., Anand, A., Sharma, N., et al. (2025). Discovering symbolic cognitive models from human and animal behavior. *bioRxiv*.
- Collins, A. G. and Frank, M. J. (2012). Cognitive control over learning: Creating, clustering, and generalizing task-set structure. *Psychological review*, 119(4):890.
- Collins, A. G. E., Ciullo, B., Frank, M. J., and Badre, D. (2017). Working memory load strengthens reward prediction errors. *The Journal of Neuroscience*, 37(16):4332–4342.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., et al. (2024). The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Ghayem, F., Meudec, R., Dockès, J., Thirion, B., and Wassermann, D. (2026). Neurocontext: Contrastive learning for neuroscience meta-analysis with rich text representation. *Imaging Neuroscience*.
- Guo, D., Shao, J., Yuan, P., Xiong, C., et al. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Kaplan, A. (1964). *The Conduct of Inquiry: Methodology for Behavioral Science*. Chandler.
- Lueckmann, J.-M., Jain, V., and Januszewski, M. (2026). Discovering mechanistic models of neural activity: System identification in an in silico zebrafish. *arXiv preprint arXiv:2602.04492*.
- Luo, X., Rechart, A., Sun, G., Nejad, K. K., et al. (2025). Large language models surpass human experts in predicting neuroscience results. *Nature Human Behaviour*.
- Luppi, A. I., Ali, H., Liu, Z.-Q., Milisav, F., et al. (2025). Cognitive cartography of mammalian brains using meta-analysis of ai experts. *bioRxiv*.
- Rmus, M., Mathony, M., Jagadish, A. K., Ludwig, T., and Schulz, E. (2025). Generating computational cognitive models using large language models. *arXiv preprint arXiv:2502.00879*.
- Ryoo, A. H.-W., Krishna, N. H., Mao, X., Azabou, M., et al. (2025). Generalizable, real-time neural decoding with hybrid state-space models. *arXiv preprint arXiv:2506.05320*.
- Tilbury, R., Kwon, D., Haydaroglu, A., Ratliff, J., et al. (2025). Ai-discovered tuning laws explain neuronal population code geometry. *bioRxiv*.
- Wairagkar, M., Card, N. S., Singer-Clark, T., Hou, X., et al. (2025). An instantaneous voice-synthesis neuroprosthesis. *Nature*.
- Wang, E. Y., Fahey, P. G., Ding, Z., Papadopoulos, S., et al. (2025). Foundation model of neural activity predicts response to new stimulus types. *Nature*.
- Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., et al. (2024). Qwen2. 5 technical report. *arXiv preprint arXiv:2412.13663*.